From a 24/7 idle server to serverless

A story of migrating Django to Lambda

and what we learned along the way

Django is the example; the lessons — adapter pattern, stateless gotchas, cost trade-offs, scaling choices — apply to any framework (Node/Express, FastAPI, Spring Cloud Function, …).

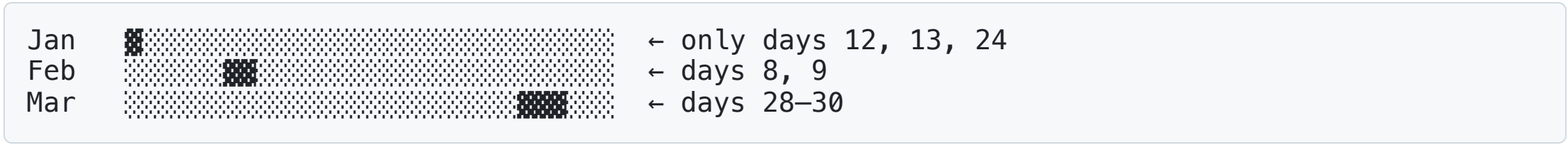
Once upon a time...

There was a project — a **B2B customer check-in app**.

- Customers signed up to use it in **bursts** (training events, conferences)
- Only **~3–5 days** of real usage per month
- The rest of the month: server running, **zero requests**

Stack: Django + Postgres on an EC2 `t3.small`, running 24/7.

The bill vs the traffic



EC2 bill: **$15/month, every month, flat.**

Actual utilization: **~5%**

> 95% of the compute spend was burned into thin air.

One day, the boss asked

> "Why are we paying the same hosting bill every month?
> Find a way to optimize this."

Hidden requirements in that one sentence:

1. Reduce compute spend
2. **No downtime for the customers actively using it**
3. **Don't rewrite the app** — it's running fine

→ Constraint #3 is the hardest: **the solution must migrate with minimal effort.**

Three roads

Option	Effort	Cost savings	Risk
(a) Scale-down EC2 → `t3.nano`	Low	~50%	Peak days may die
(b) Cron stop/start EC2 by schedule	Medium	~70%	Depends on correct schedule; cold-boot downtime
(c) **Serverless** — pay only when there's a request	?	~95%	Need to learn Lambda + rewrite the gateway

Option (c) has the most potential — but "effort = ?" needs clarifying.

Why serverless won

Pricing model (Singapore, ap-southeast-1):

```
Lambda           = #requests × $0.20/1M
                 + GB-s × $0.0000167 (x86) / $0.0000133 (arm64, -20%)
API Gateway HTTP = #requests × $1.25/1M
```

Always-free tier per month — never expires, applies globally:

- 1,000,000 Lambda requests
- 400,000 GB-seconds
- 1,000,000 API Gateway HTTP requests (first 12 months only)

Check-in project: **~30K requests/month → $0** (well inside free tier).

Source: AWS Public Pricing API, effective 2025-11-01.

But… how does Django even run on Lambda?

Lambda receives a **JSON event** from API Gateway.

Django speaks **WSGI/ASGI** (scope dict + send/receive coroutines).

→ **The two sides don't understand each other.**

We need an **adapter** in the middle — not a rewrite, just a translation layer.

Mangum — the 5-line "magic"

```python
# config/handler.py
import os
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

from mangum import Mangum
from config.asgi import application

handler = Mangum(application, lifespan="off")
```

That's the entire "Lambda-fy Django" step. Models, views, URLs, settings — unchanged.

Mental model — don't conflate

Container (EC2/ECS)	Lambda
Process runs 24/7	Process runs per request
You manage RAM/CPU	Pick RAM, CPU scales with it
Long-lived state OK	**Stateless required**
Scale = add tasks	Scale = AWS spawns contexts
Cold start = deploy time	Cold start = every new container

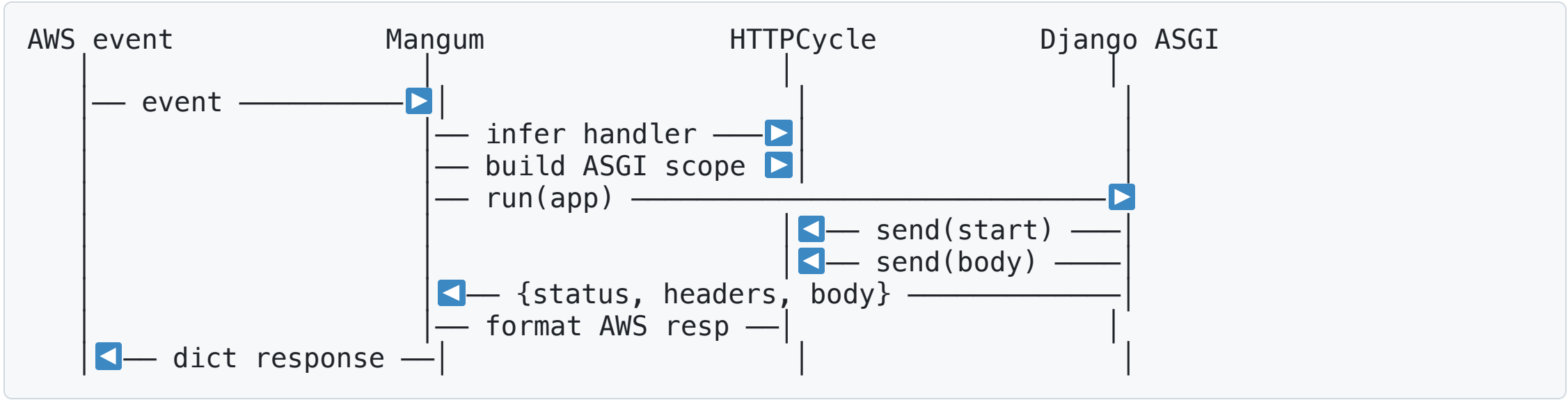
> **One sentence to remember:** *"Containers run forever, Lambda runs per request."*

Cold start — real numbers from the demo

```
Cold-start Firecracker VM creation          ~100–200ms
+ Extract zip into /var/task                 ~50ms
+ Import handler module                      1000–1500ms
  ├─ Import Django, DRF, Mangum, boto3
  ├─ Set up settings, middleware chain
  └─ Initialize ASGI app
+ Handle the request                         ~50–200ms
──────────────────────────────────────────
Total cold start:                            ~1.5s
Subsequent warm requests:                    ~50ms
```

Measured on this demo, M-series, Python 3.12 arm64.

How Mangum works inside



Three layers: **adapter → cycle → app.** Each is ~100 lines. One evening of reading and you understand the whole thing.

Inside the Lambda container

```
/var/task/      ← your code (extracted from S3 zip)
/var/runtime/   ← Python interpreter + AWS SDK
/opt/           ← Lambda Layers mount point
/tmp/           ← writable, 512MB–10GB, dies with container
```

Auto-set environment variables — use these instead of hardcoding:

Variable	Value
`LAMBDA_TASK_ROOT`	`/var/task`
`AWS_LAMBDA_FUNCTION_NAME`	`workshop-todo-api`
`AWS_REGION`	`us-west-2`
`AWS_LAMBDA_INITIALIZATION_TYPE`	`on-demand` / `provisioned-concurrency`

All `LAMBDA_*` and `AWS_*` vars are set by the runtime — no config needed.

Handler path: where Python looks

```
/var/task/
├── todo_api/
│   ├── config/handler.py    ← Handler points here
│   └── todos/
```

Default `sys.path = [/var/task/]` → `import config.handler` **fails**.

Two ways to fix:

Option	`Handler:` value	`PYTHONPATH`
Long handler path	`todo_api.config.handler.handler`	*(not set)*
Short handler + path	`config.handler.handler`	`/var/task/todo_api`

Pattern applies to **any nested Python project** on Lambda.

Stateless — Django gotchas

Django concept	The problem on Lambda	Fix
`MEDIA_ROOT` upload	`/tmp` is ephemeral, 512 MB	S3 + `django-storages`
`collectstatic`	No nginx to serve static	S3 + CloudFront
DB connections	Cold start = new connection	**RDS Proxy** or `CONN_MAX_AGE=0`
Migrations	Where do they run?	CI/CD step before deploy
Celery workers	Worker = long-lived	SQS + Lambda consumer
Sessions	In-memory dies between invocations	ElastiCache or DB sessions
WebSockets	HTTP API doesn't support them	Separate API GW WebSocket API

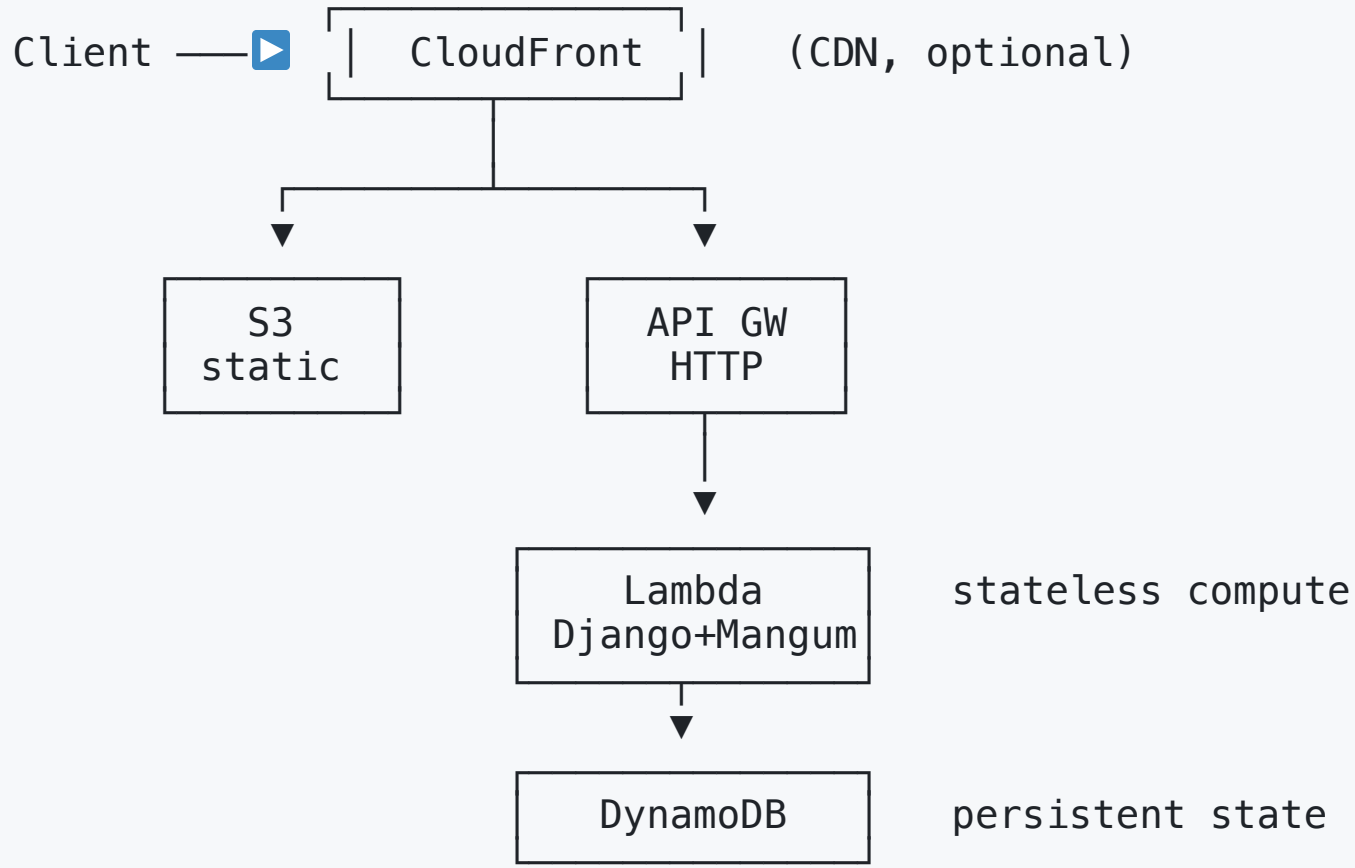
Where does state go? → DynamoDB

The demo uses DynamoDB instead of SQLite. Why?

- **Serverless billing** — PAY_PER_REQUEST, no steady RCU/WCU cost
- **Same philosophy** as Lambda: idle = $0
- **IAM scope-tight** via SAM's `DynamoDBCrudPolicy` (this one table, not full DynamoDB access)
- The audience can *see* it: **state persists across cold starts**

```
# todos/store.py — boto3 directly, no ORM
table.put_item(Item={"id": str(uuid4()), "title": ..., "done": False})
```

The final architecture



Three cost choices baked in: **HTTP API** · **arm64** · **PAY_PER_REQUEST**.

Live demo — 10 minutes

We'll go through:

1. Open `todos/store.py` — the boto3 layer (30s)
2. Open `config/handler.py` — 5 lines of Mangum (30s)
3. Open `template.yaml` — `arm64`, `HttpApi`, `DynamoDBCrudPolicy` (1 min)
4. Run `./scripts/02-deploy.sh` — real deploy (3 min)
5. **Open the live URL in browser** — cold-start spinner, add/toggle/delete todos (2 min)
6. Refresh → warm hit, latency badge drops 10× (30s)
7. Run `./scripts/03-test.sh` — curl proof state lives in DynamoDB (1 min)
8. CloudWatch tab → `INIT_START` + `REPORT … Init Duration` (1 min)

Cost reality — workshop scenario

```
1M requests/month, 200ms each @ 1024MB, arm64, Singapore:

Lambda compute   = 1M × 0.2s × 1GB × $0.0000133  =  $2.67
Lambda requests  = 1M × $0.20/1M                 =  $0.20
API GW HTTP      = 1M × $1.25/1M                 =  $1.25
DynamoDB writes  = 1M × $0.71/1M                 =  $0.71
─────────────────────────────────────────────────────────
Total                                            ≈  $4.83/month
```

Same workload on the smallest Fargate task + ALB in Singapore: **~$28/month**.

The actual check-in project (~30K req/month): **$0** (free tier covers it).

Source: AWS Public Pricing API, ap-southeast-1, effective 2025-11-01.

But it doesn't always win

Traffic	Serverless	Fargate + ALB	Verdict
100K req/month	~$0.50	~$28	Serverless ~55× cheaper
1M	~$5	~$28	Serverless ~6× cheaper
10M	~$50	~$30–45	**Tie**
100M	~$500	~$80 (cluster)	**Containers win**
1B	~$5,000	~$400	Containers win by a lot

> **Serverless wins on low/bursty traffic.**
> **Steady, high traffic → containers win.**
> **Don't choose by trend. Choose by profile.**

Singapore region, smallest steady Fargate task + ALB base. Real numbers vary with task size, RPS, and reserved-capacity discounts.

Top 5 cost optimizations

1. **ARM / Graviton2** — one config line, instant 20% savings
2. **Right-size memory** — use lambda-power-tuning
3. **HTTP API** instead of REST API — 70% cheaper, fits 90% of use cases
4. **CloudWatch Logs retention** — set 14–30 days, never "never expire"
5. **Reserved Concurrency** — guardrail against runaway cost (attack, loop)

Items 1 and 3: fix today. One IaC line each.

> **For W5 MH5 — pick one scaling pattern:**
> **Reserved Concurrency** (#5 above) · **Provisioned Concurrency** (warm pre-init) · **Async + DLQ** (failed-event capture) · **S3-event-triggered Lambda** (pipeline)

Take-aways

1. Serverless **isn't free** — it **doesn't cost when idle**.
2. Mental model: **"per request"** — every gotcha traces back here.
3. Choose by **traffic profile**, not by trend.
4. **Read the lib's source** before adopting — Mangum is 10 files, one evening.
5. **Test, don't guess** — can you drop `auth`? Measure, don't assume.

> **"Containers run forever, Lambda runs per request."**

Source code

`github.com/TechX-Corp/xbrain-w5-byol-python-django`

Django + Mangum + DynamoDB · SAM template · scripts · UI

Q&A

The dumbest question gets the most serious answer.